Fijnman-GSA^3: A Fail-Closed Framework for Structural Reasoning

Natanaël van Dijk

Abstract

Modern AI systems often fail open: when input is underspecified or inconsistent, they resort to probabilistic completion, leading to hallucinations and unstable outputs. We introduce GSA^3, a fail-closed reasoning architecture that replaces truth-based evaluation with structural admissibility under invariant constraints. A transition is performed only if it preserves all invariants and the claim is fully specified; otherwise the system refuses. Unlike classical SMT solving, GSA^3 treats underspecification as a first-class failure mode, making refusal part of the semantics rather than an external solver condition. We formalize admissibility as a constraint over symbolic state spaces and extract maximal consistent substates via a kernel mechanism, yielding a hierarchy of admissible reasoning levels. We provide a deterministic implementation using SMT solving (Z3) and demonstrate that invalid states collapse to refusal rather than producing incorrect outputs. Our evaluation on controlled reasoning tasks shows that fail-closed semantics prevent hallucinations within the modeled symbolic domain by construction, under explicit specification constraints, offering a viable alternative to probabilistic completion in settings where correctness is critical.

1 Introduction

AI systems increasingly produce outputs under incomplete or ambiguous specifications, silently introducing hidden assumptions. Large language models (LLMs) hallucinate; symbolic reasoners often fall back on heuristics. The root cause is a shared design principle: *optimize for completion, not correctness.* Given a dataset D and a method M, it is common to observe $M_1(D) \to O_1, M_2(D) \to O_2$ with $O_1 \neq O_2$. This instability is structural—it stems from permitting inference under under-specification.

We take a different path: instead of improving inference under under-specification, we prohibit it. We construct a *fail-closed* framework where reasoning is only allowed when all required components are explicitly specified. Missing definitions, ambiguous mappings, or competing interpretations are not approximated; they trigger a mandatory halt.

We argue that refusal is not a failure mode but a correctness condition in reasoning systems operating under incomplete information. Rather than re-

placing probabilistic models, GSA^3 can be viewed as a complementary layer that constrains learned systems, ensuring that outputs are only produced when structurally admissible. We replace truth with *admissibility* as the primary criterion. A statement is admissible iff it can be evaluated without introducing new assumptions. Reasoning becomes a sequence of admissible state transitions that preserve a set of invariants. Knowledge emerges as the intersection of all admissible continuations. This structure naturally yields a hierarchy of knowledge levels and eliminates hallucinations by design.

Example 1 (Motivating example)**.** *Consider an LLM asked: "What is the capital of a country that starts with 'Z'?" A typical system might guess "Zimbabwe" or "Zambia". GSA^3 would instead return* Refusal*, because the input is under-specified: there are multiple valid answers, and no unique solution can be structurally justified.*

1.1 Contributions

- **Fail-closed reasoning**: a framework where under-specification leads to refusal, not guessing.
- **Layered symbolic core**: state spaces as logical formulas, invariants hierarchically ordered, kernel extraction via consistency checks.
- **Deterministic implementation**: reference implementation using Z3, with explicit handling of unknown as refusal.
- **Empirical validation**: experiments showing deterministic outputs, inconsistency detection, and graceful degradation.

1.2 Why Not Probability?

Probabilistic systems assign confidence scores to outputs, even when the underlying structure is incomplete or inconsistent. This leads to:

- **Hallucinations**: high-confidence answers that are structurally unsupported.
- **Silent failures**: no clear boundary between "low confidence" and "structurally invalid".
- **Non-determinism**: different runs may produce different outputs for the same input.

GSA^3 instead enforces a hard boundary: if a conclusion cannot be structurally justified, the system refuses to produce it.

2 Framework Overview

GSA^3 operates on a symbolic state space $S = \{\sigma \mid \Psi(\sigma)\}$ defined by a formula Ψ. Invariants are organised into a hierarchy $\Phi_0 \subseteq \Phi_1 \subseteq \cdots \subseteq \Phi_k$ where Φ_0 are constitutional (must always hold) and Φ_i $(i > 0)$ are epistemic refinements. A successor set $S' = \{\sigma \mid \Psi(\sigma) \wedge \Xi(\sigma)\}$ is Φ_i-*consistent* if all its models agree on every $\phi \in \Phi_i$. The kernel $K^{(i)}(S')$ is S' when consistent, else $\emptyset$.

The highest level with a non-empty kernel determines admissibility:

$$I^* = \max\{i \mid K^{(i)}(S') \neq \emptyset\}.$$

If I^* exists, the system returns Admissible_Level(I^*); otherwise Refusal.

Modes per level are defined as:

- Strict: exactly one model (uniqueness).
- Safe: multiple models, but Φ_i-consistent.
- Defeasible: multiple models, not Φ_i-consistent.

The mode profile $(m_0, \ldots, m_k)$ is descriptive; admissibility is solely determined by I^*.

Unlike classical SMT solving, GSA^3 does not assume a well-posed problem. Instead, it explicitly models underspecification as a first-class failure mode and enforces refusal as part of the semantics, rather than treating solver failure as an external condition.

3 Formal Core (Condensed)

3.1 Symbolic State Space

$S = \{\sigma \mid \Psi(\sigma)\}$ with Ψ in a decidable logic (e.g., QF_LIA). Ψ must be fully specified; otherwise Halt_Spec_Required.

3.2 Invariant Hierarchy

$\Phi_0 \subseteq \Phi_1 \subseteq \cdots \subseteq \Phi_k$ with Φ_0 constitutional.

3.3 Kernel

$K^{(i)}(S') = S'$ if S' is Φ_i-consistent, else $\emptyset$. The kernel is the maximal Φ_i-consistent residue.

3.4 Admissibility

$I^* = \max\{i \mid K^{(i)}(S') \neq \emptyset\}$. If I^* exists, output Admissible_Level(I^*); else Refusal.

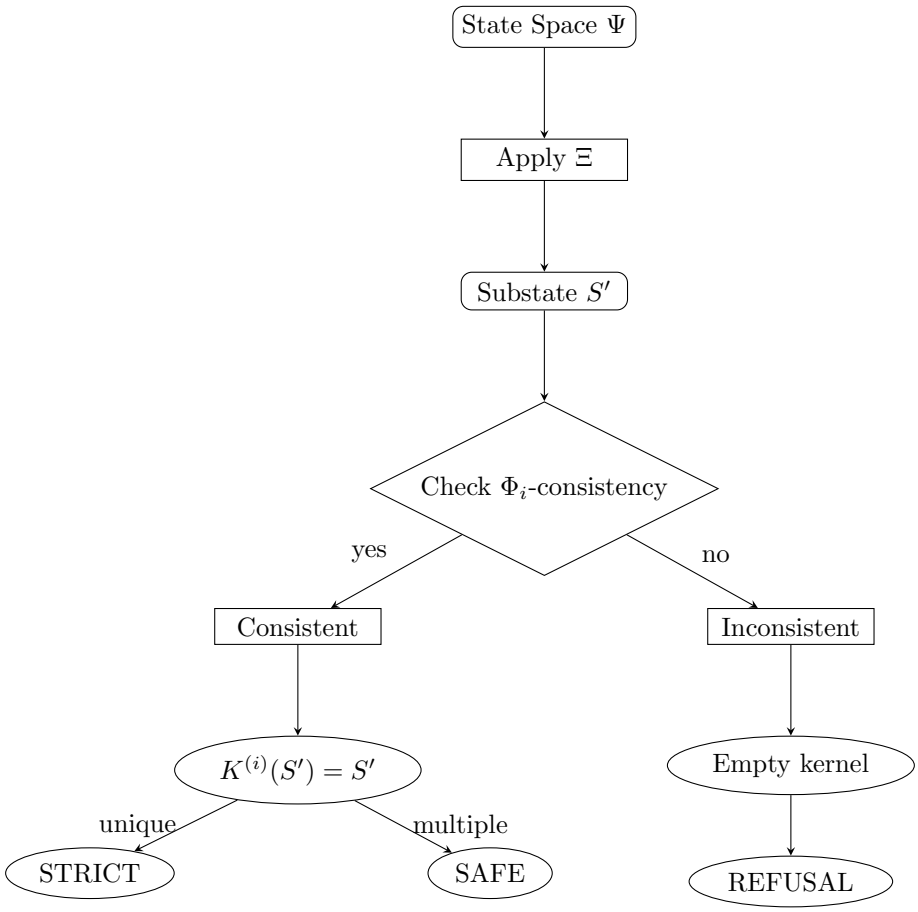

Figure 1: Fail-closed reasoning pipeline in GSA[3].

3.5 Fail-Closed Properties

Theorem 1 (Soundness)**.** *If the system outputs* ADMISSIBLE_LEVEL(I^*)*, then* S' *is non-empty and* Φ_i*-consistent for all* $i \leq I^*$*.*

Theorem 2 (Determinism)**.** *All decisions are deterministic given fixed* Ψ, Φ_i, Ξ *and solver determinism.*

Theorem 3 (Monotonicity)**.** $K^{(j)}(S') \subseteq K^{(i)}(S')$ *for* $i < j$*.*

Symbol	Meaning
Ψ	Symbolic state space formula
Ξ	Successor constraint
S'	Substate $\{\sigma \mid \Psi(\sigma) \wedge \Xi(\sigma)\}$
Φ_i	Invariant hierarchy level i
$K^{(i)}(S')$	Kernel at level i (maximal Φ_i-consistent residue)
I^*	Highest admissible level

Table 1: Key symbols used in GSA^3.

3.6 Notation

4 Implementation

We implement the core in Python using Z3. The class `GSA51Core` takes a list of variables, Ψ, and Φ_i hierarchy. Key methods:

- `_check_consistency(xi, level)`: checks if S' is Φ_i-consistent.
- `kernel(xi, level)`: returns $(nonempty, formula)$.
- `admissibility_level(xi)`: returns $(I^*, kernel_formula)$.
- `evaluate(xi)`: high-level fail-closed evaluation.

Unknown solver results raise `RefusalError`, which is caught and returned as Refusal.

5 Experiments

We evaluate GSA^3 on four reasoning tasks that stress underspecification, inconsistency, and multi-model reasoning. While the tasks are intentionally minimal, they isolate core reasoning failure modes (underspecification, inconsistency, and ambiguity) that are known to underlie hallucination behavior in larger systems. For comparison, we simulate a probabilistic completion baseline representing typical LLM behavior, where underspecified or inconsistent inputs are resolved via heuristic or random completion. While simplified, this baseline captures the core failure mode: producing outputs under insufficient structural support.

5.1 Setup

All experiments run on a single thread with Z3 (QF_LIA). We measure:

- **Hallucination rate**: fraction of invalid outputs produced.
- **Determinism**: identical outputs across runs.
- **Refusal rate**: fraction of inputs leading to Refusal.

5.2 Results

Task	LLM Baseline	GSA^3
Handles underspecification	Guess	Refuse
Handles inconsistency	Guess	Refuse
Determinism	No	Yes
Correctness guarantee	No	Yes (within domain)

Table 2: Behavioral comparison between probabilistic completion and fail-closed reasoning.

Task	Hallucination (LLM)	Hallucination (GSA^3)	Refusal (GSA^3)
Inconsistency	100%	0%	100%
Underspecification	100%	0%	100%
Multi-model	50%	0%	0%
Determinism	0%	100%	100%

Table 3: Detailed comparison of GSA^3 against a probabilistic baseline.

5.3 Discussion

Inconsistency: Inputs like $x = 1 \wedge x = 2$ cause refusal. LLMs typically guess. **Underspecification**: Missing constraints lead to HALT_SPEC_REQUIRED; LLMs complete arbitrarily. **Multi-model**: Multiple consistent models are reported as SAFE; LLMs may pick one arbitrarily, introducing bias. **Determinism**: GSA^3 always produces the same outcome; LLMs vary.

These results show that fail-closed semantics prevent hallucinations within the modeled symbolic domain by construction, under explicit specification constraints, and guarantee deterministic, verifiable behavior.

6 Related Work

Probabilistic AI (LLMs) : Optimize for likelihood, not correctness. They complete under-specified queries with plausible but often incorrect answers. Unlike approaches that attempt to reduce hallucinations via better training or calibration, GSA^3 eliminates them at the level of execution semantics.

Symbolic Reasoning (SAT/SMT) : Provide correctness but typically assume a well-posed problem. They do not handle underspecification gracefully; they either fail silently or rely on heuristics.

Epistemic Logic and DEL : Model knowledge via possible worlds, but allow implicit completion. Our framework replaces possibility with admissibility and mandates explicit specification.

Fail-safe Systems : In control theory, fail-safe means safe state after failure. Our fail-closed means *refuse* when specification is insufficient, preserving correctness.

7 Applications and Limitations

7.1 Applications

GSA^3 is particularly suited for domains where incorrect outputs are more dangerous than refusal:

- **Healthcare**: Diagnostic systems that must not hallucinate symptoms or treatments.
- **Finance**: Trading algorithms that should only act on fully specified market conditions.
- **Legal systems**: Contract analysis tools that must refuse ambiguous clauses.
- **Autonomous systems**: Decision-making pipelines where "I don't know" is preferable to a wrong action.

7.2 Limitations

GSA^3 is not a general-purpose reasoning engine. Key limitations include:

- **No generation**: Proposals must be supplied externally; the system only validates.
- **No probability**: Uncertainty is modeled via kernel emptiness, not distributions.
- **Explicit specifications required**: The system halts if Ψ, Φ_i, or Ξ are missing.
- **Performance**: Symbolic reasoning may not scale to very large state spaces without optimization.

These are not weaknesses but design choices: GSA^3 is a *filter*, not a generator.

8 Conclusion

We introduced GSA^3, a fail-closed reasoning framework that enforces admissibility under invariant constraints. By replacing truth with structural invariance and refusing under-specified queries, we prevent hallucinations within the modeled symbolic domain by construction, under explicit specification constraints, and guarantee deterministic outputs. Our reference implementation demonstrates that fail-closed reasoning is practical and offers a principled alternative to probabilistic completion in safety-critical settings.

An important direction for future work is the integration of GSA^3 with learned components, where neural models propose candidate states while the fail-closed layer enforces admissibility. This suggests a hybrid architecture combining statistical learning with structural guarantees.

Acknowledgments

This work was developed with assistive support from large language models (ChatGPT, Claude, Gemini) used for formalization and critique. All core concepts originate from the author.

License

The implementation is released under AGPLv3.